

Sudoku Cube:

Description of data structure:

The data structure I chose is a simple nested array for each face of the cube. Each face contains an array of size 3, representing rows, and inside each index another array of size three. Hence a 2D table more or less. I chose this type of data structure because I am very familiar with it, and I feel like its easier to index and manage.

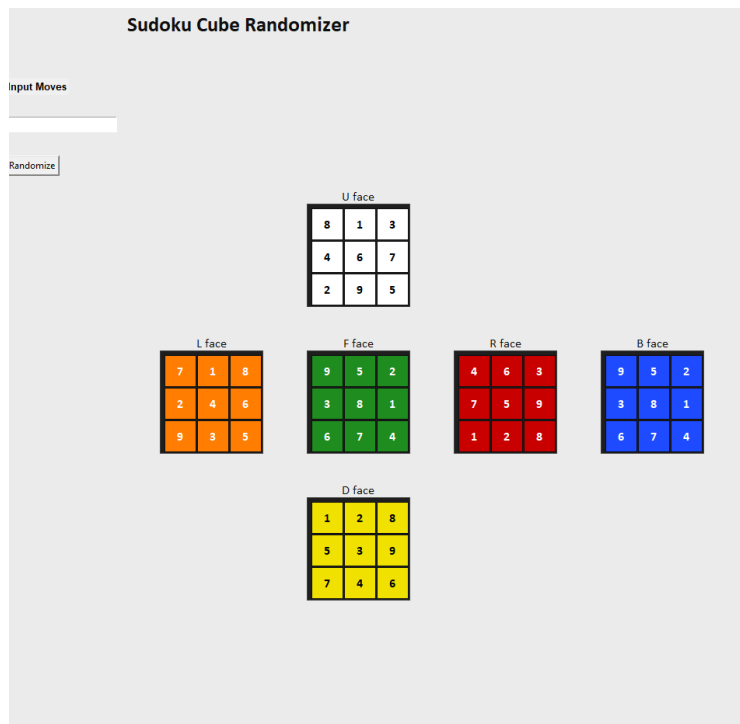
Example of data structure:

This is a screenshot of how I set up my faces at the start of the program.

```
CUBE = {
  "U": [ # Up face
    [8, 1, 3],
    [4, 6, 7],
    [2, 9, 5]
  ],
  "L": [ # Left face
    [7, 1, 8],
    [2, 4, 6],
    [9, 3, 5]
  ],
  "F": [ # Front face
    [9, 5, 2],
    [3, 8, 1],
    [6, 7, 4]
  ],
  "R": [ # Right face
    [4, 6, 3],
    [7, 5, 9],
    [1, 2, 8]
  ],
  "B": [ # Back face
    [9, 5, 2],
    [3, 8, 1],
    [6, 7, 4]
  ],
  "D": [ # Down face
    [1, 2, 8],
    [5, 3, 9],
    [7, 4, 6]
  ]
}
```

Example of GUI output: (Done with the help of ChatGPT)

Prompt for chat gpt: 'Produce a gui using tkinter for a sudoku cube, the GUI could be very simple, perhaps with the cube unfolded, Plain text is fine.'



Description of Randomizer:

For the randomizer I use the library random to when the user clicks on the button "Randomize" it triggers a function called `randomize_cube`, which for the number of moves in input, it give a "random" face and direction in which to rotate. Then it calls the function `rotate`, which first rotates the face itself, and then the adjacent strips using just if statement. The way it rotates is by indexing the nested arrays, with logic. The logic is thought of for a 3D object of course, and then the UI is refreshed to the new faces.

Code for randomizer: attached in submission in file named "[sudokuCube.py](#)". To run just press `Fn + F5` or a running button in vscode. The UI will pop up, and to start the scrambling you simple input the number of moves and press the "Randomize" button. After that the UI sudoku cube should be scrambled.

Example for randomize code:

```
def randomize_cube(cube, num_moves):
    faces = list(cube.keys())
    directions = ["Clockwise", "Counterclockwise"]

    for i in range(num_moves):
        f = random.choice(faces)
        d = random.choice(directions)
        rotate(cube, f, d)
    return cube
```

Heuristic: For the heuristic I thought to do : count the number of cubbies that are not in the correct face. More in depth, for example if one of the faces say we need numbers 1-9 unique numbers, then if we have two 2's, and two 9's, then we have at least two objects misplaced. I think is admissible because it doesn't overestimate the number of steps to get the misplaced items. It takes at least one rotation to have a misplaced number , and in most cases is always more than 1 misplaced items because of the adjacent strips. Also is very simple to count since we just have to count them once the scrambling is done. I applied it by a simple function that counts which numbers are in a single face and adds the missing numbers count to a variable. I printed it before scrambling and after scrambling. The heuristic is automatically calculated when you submit the number of moves and the output in in the output window of vs code.

Function:

```
def heuristic(cube):
    misplaced = 0
    target = set(range(1, 10))

    for face in cube.values():
        face_numbers = {num for row in face for num in row}
        # Count how many numbers are missing from this face
        missing = target - face_numbers
        misplaced += len(missing)

    return misplaced
```

Output:

```
Heuristic at start: 0
Heuristic after scramble: 9
□
```

What I learned:

From this project I learned about making big problems simpler. I started out with a very complicated data structure in my opinion (maps) and although very high level and useful in many things, I am more comfortable in arrays, and when i implemented them into the randomizer it was longer, but equally effective. I was overcomplicating stuff. Same with logic for the rotation of the faces. I learned even more about

indexing 2D arrays. I also learned a very cool library called tkinter that is very intuitive and easy to use and create UIs.